UniFlow Smart Campus

Dossier technique – Validation 1

Plateformedegestionuniversitaireenarchitecturemicroservices

Module AWD – ESPRIT – Année universitaire 2026-2027
Équipe : Feriel · Haifa · Lina · Ferdaws · Syrine · Farah
DépôtGit : https://github.com/ferielyahia/UniFlow-Smart-Campus-4SSA16.git

1. Description du sujet

UniFlow Smart Campus est uneplateforme de gestion universitaire qui coordonne plusieurs activités d'un campus : les étudiants et leurs informations académiques, les cours et les emplois du temps, les présences, les salles et la vie universitaire.

Problématique

Comment fiabiliser une gestion universitaire répartie entre plusieurs outils ? Les informations sont dispersées, la synchronisation est difficile, il existe un risque d'incohérence entre les données et la visibilité sur les activités est limitée.

Objectif

Concevoir une plateforme modulaire, évolutive et fiable pour les différents services d'un campus, en gardant des responsabilités claires grâce à une architecture microservices.

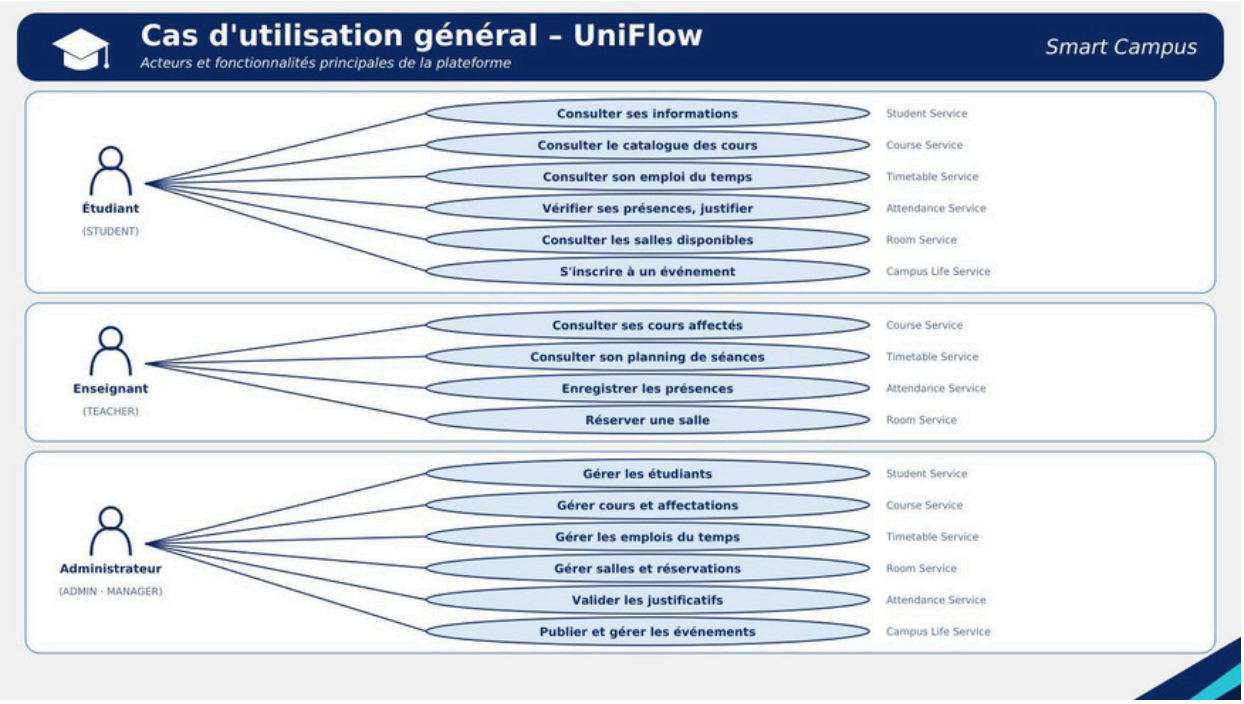
2. Acteurs et fonctionnalités

Acteur		Besoins principaux
Étudiant	STUDENT	Consulter ses informations, les cours, son emploi du temps, ses présences, les salles ; s'inscrire aux événements
Enseignant	TEACHER	Consulter ses cours et son planning, enregistrer les présences, réserver une salle
Administrateur	ADMIN	Gérer les étudiants, les cours, les emplois du temps, les salles ; valider les justificatifs ; publier les événements

Fonctionnalités principales

- Gestion et consultation des informations des étudiants (Student Service)
- Gestion du catalogue des cours et des affectations d'enseignants (Course Service)
- Organisation et consultation des emplois du temps (Timetable Service)
- Suivi des présences, absences et justificatifs (Attendance Service)
- Gestion des salles, des équipements et des réservations (Room Service)
- Publication d'événements universitaires et inscriptions (Campus Life Service)

Diagramme de cas d'utilisation général



3. Découpage en microservices

Chaque microservicecorrespond à unecapacité métier (contexte borné) et possède ses propres données. Ledécoupage ne suit ni les tablesni les écrans.

Microservice	Responsabilité	Responsable
Student Service	Informations et gestion des étudiants	Feriel
Course Service	Catalogue et gestion des cours	Haifa
Timetable Service	Organisation des emplois du temps	Lina
Attendance Service	Suivi des présences	Ferdaws
Room Service	Gestion des salles	Syrine
Campus Life Service	Activités et informations de la vie universitaire	Farah

4. Données gérées par chaque service

Service	Entités	Base de données
Student	Student, AcademicYear, Enrollment	MySQL
Course	Subject, Course, TeacherAssignment	H2
Timetable	Timetable, TimeSlot, Session	H2
Attendance	AttendanceSheet, AttendanceRecord, Justification	H2
Room	Room, Equipment, Reservation	MySQL
Campus Life	Event, Registration	MySQL

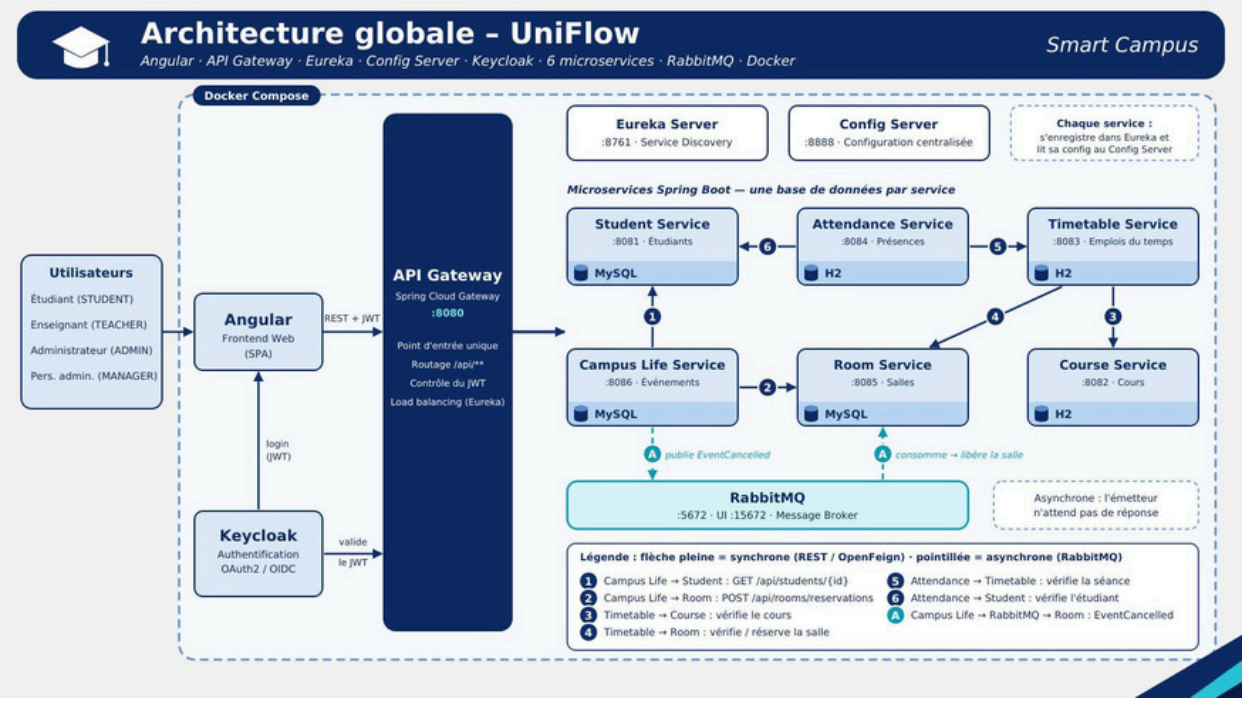
Règle:unebase par microservice, aucunebasepartagée. Les références entre services (studentId,courseId, roomId, teacherId…)sontdesimples identifiants sans clé étrangère, car un servicen'accède jamais directement à labased'unautre.

5. Interactions entre les services

Appelant → appelé	Mode	Rôle
Campus Life → Student	Synchrone (OpenFeign)	Vérifier l'étudiant à l'inscription
Campus Life → Room	Synchrone (OpenFeign), optionnel	Réserver la salle d'un événement
Timetable → Course	Synchrone (OpenFeign)	Vérifier le cours
Timetable → Room	Synchrone (OpenFeign)	Vérifier / réserver la salle
Attendance → Timetable	Synchrone (OpenFeign)	Vérifier la séance
Attendance → Student	Synchrone (OpenFeign)	Vérifier l'étudiant
Campus Life → Room	Asynchrone (RabbitMQ)	Événement EventCancelled : libérer la salle

Quand un service appelé est indisponible, l'appelantattend un délai maximal puisrenvoie une réponse contrôlée (503). Pour l'asynchrone, le message attend dans le broker jusqu'à ce que le consommateurredevienne disponible.

6. Architecture globale



Composant	Rôle
Angular (frontend)	Application cliente, appelle l'API via la Gateway
API Gateway (Spring Cloud Gateway, :8080)	Point d'entrée unique, routage /api/**, contrôle du JWT
Eureka Server (:8761)	Annuaire des services (découverte)
Config Server (:8888)	Configuration centralisée des services
Keycloak	Authentification et rôles (OAuth2 / OIDC, JWT)
RabbitMQ (:5672)	Broker de messages pour la communication asynchrone

Composant	Rôle
Docker / Docker Compose	Conteneurisation et lancement de l'ensemble

7. Technologies

- Frontend : Angular
- Backend : Java, Spring Boot, Spring Data JPA, OpenFeign
- Infrastructure : Spring Cloud Gateway, Eureka, Config Server
- Sécurité : Keycloak (OAuth2 / OIDC, JWT)
- Messagerie : RabbitMQ
- Bases de données : MySQL, H2
- Déploiement : Docker, Docker Compose
- Documentation d'API : Swagger / OpenAPI
- Outils : Git / GitHub, Trello

8. Organisation du projet

Dépôt Git

- Backend/ : un projet par microservice
- Frontend/ : application Angular
- Documentation/Presentation/ et Documentation/Diagrams/ : présentation et diagrammes
- Documentation/ : dossier technique
- README.md : sujet, équipe, architecture, lancement
- Branche main et une branche feature/<service> par membre, Pull Request vers main, commits réguliers
- L'enseignant est ajouté comme collaborateur du dépôt

Gestion de projet

L'équipe utilise Trello : tableau À faire / En cours / Terminé, avec une carte par tâche et par service.

Répartition des tâches

Membre	Microservice
Feriel	Student Service
Haifa	Course Service
Lina	Timetable Service
Ferdaws	Attendance Service
Syrine	Room Service
Farah	Campus Life Service

Tâches communes (réalisées en équipe) : l'API Gateway, Eureka, le Config Server, Keycloak, Docker Compose et le Frontend Angular

Planning prévisionnel

- Conception et dépôt Git (validation 1)
- Création des projets et de l'infrastructure

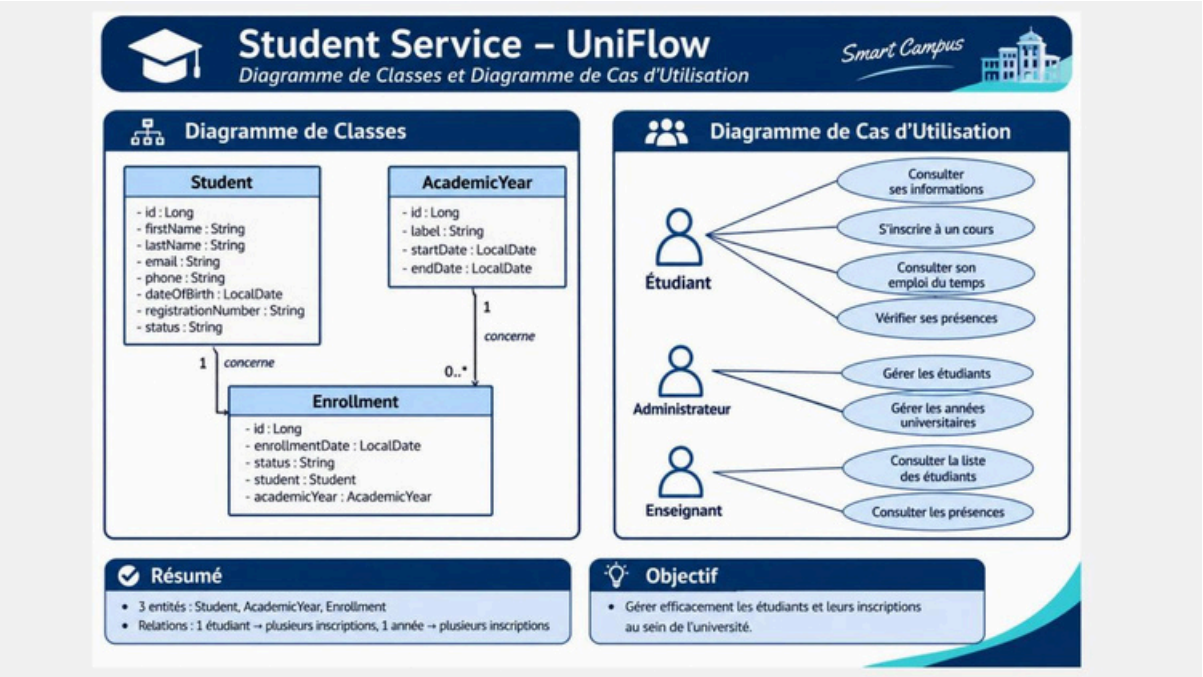
- Développement des services
- Intégration avec Docker Compose
- Tests et démonstration finale

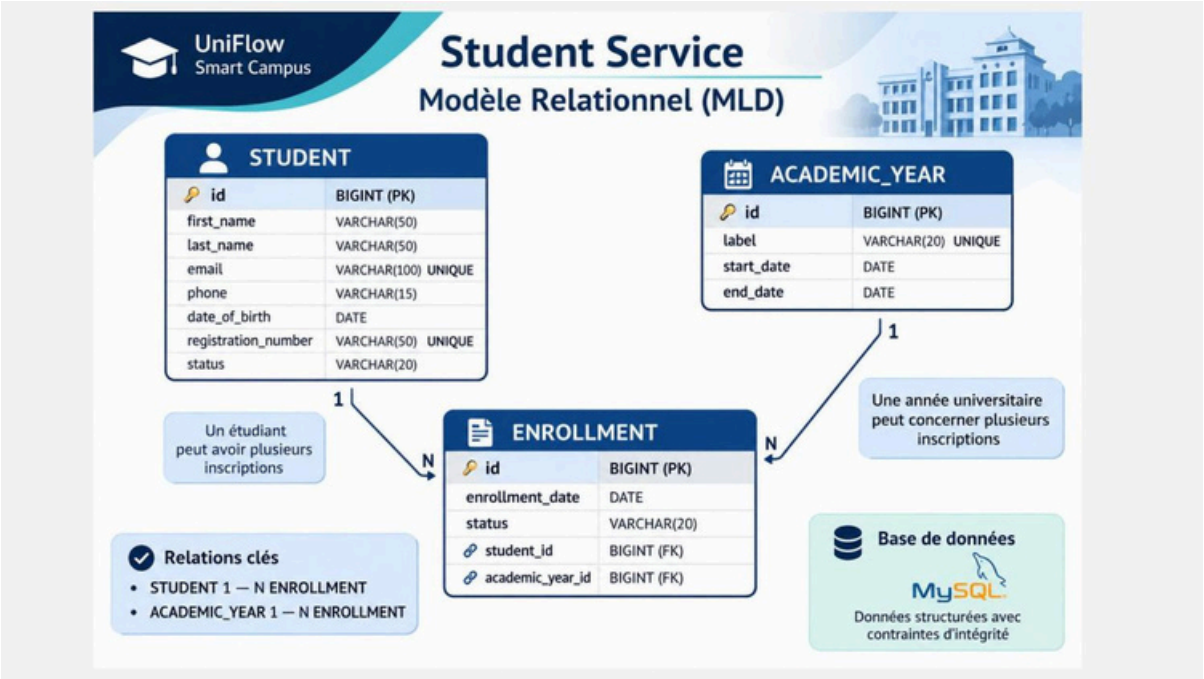
Risques

Risque	Mesure
Service indisponible	Délai d'attente puis réponse 503
Contrats d'API différents	Swagger et tableau d'API commun
Intégration tardive	Docker Compose dès le début
Retards et conflits Git	Suivi Trello et branches par service

9. Conception détaillée des microservices

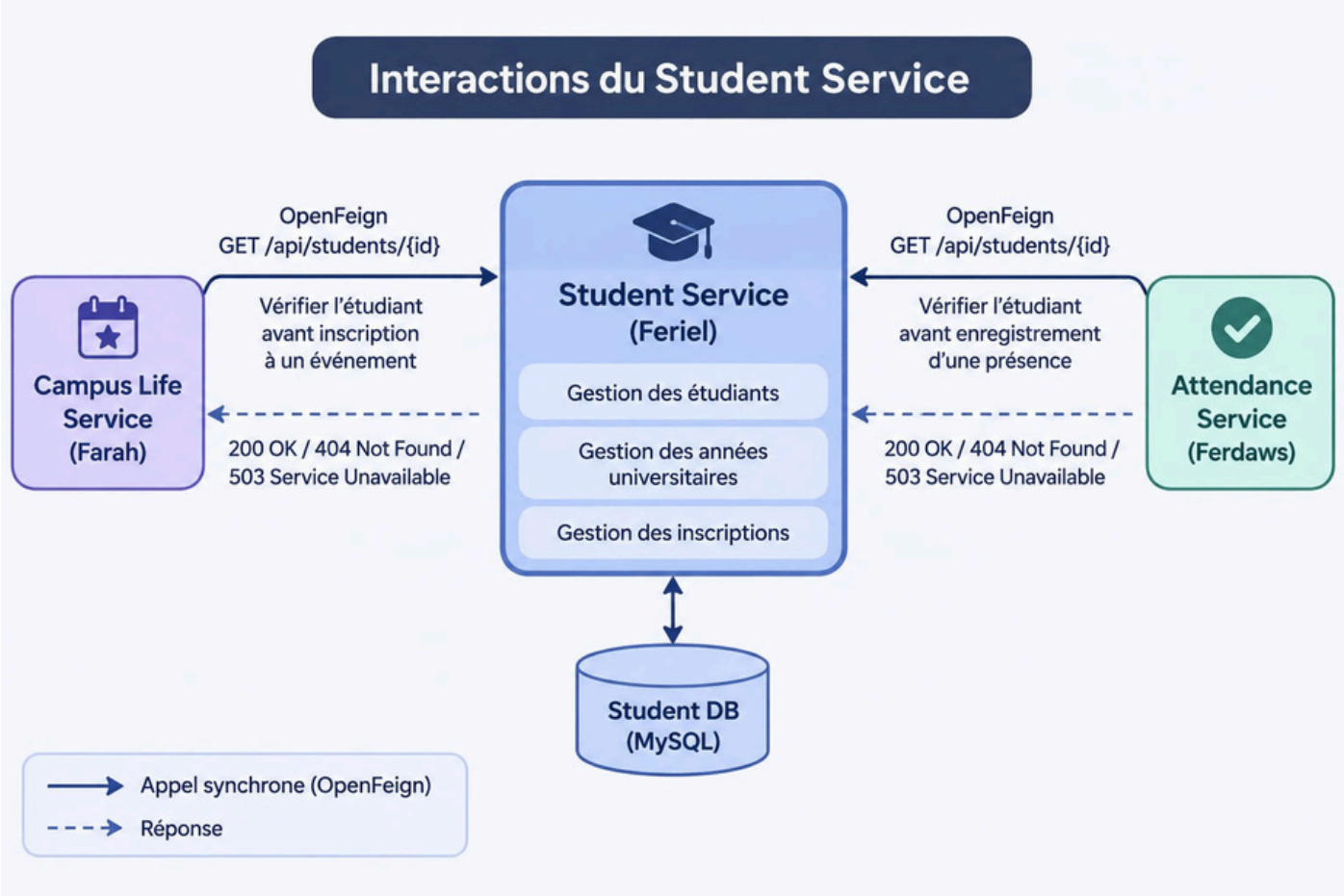
9.1 Student Service





Méthode et URL	Rôle	Description	Codes
GET /api/students	ADMIN, TEACHER	Liste des étudiants	200
GET /api/students/{id}	ADMIN, TEACHER, STUDENT	Détail d'un étudiant	200, 404
POST /api/students	ADMIN	Ajouter un étudiant	201, 400, 409
PUT /api/students/{id}	ADMIN	Modifier les informations d'un étudiant	200, 400, 404, 409
DELETE /api/students/{id}	ADMIN	Supprimer un étudiant	204, 404, 409
GET /api/academic-years	ADMIN, STUDENT	Liste des années universitaires	200
GET /api/academic-years/{id}	ADMIN, STUDENT	Détail d'une année universitaire	200, 404
POST /api/academic-years	ADMIN	Ajouter une année universitaire	201, 400, 409
PUT /api/academic-years/{id}	ADMIN	Modifier une année universitaire	200, 400, 404
DELETE /api/academic-years/{id}	ADMIN	Supprimer une année universitaire	204, 404, 409
GET /api/enrollments	ADMIN	Liste des inscriptions	200

Interactions



Synchrone vers Course Service et Attendance Service (OpenFeign)

Lorsqu'un administrateur souhaite consulter ou mettre à jour le profil d'un étudiant, le Student Service peut communiquer avec les autres microservices afin de récupérer certaines informations liées à son parcours académique.

A. Vérification auprès du Course Service

Le Student Service peut appeler le Course Service via :

GET /api/courses/student/{studentId}

Cet appel permet de récupérer les cours auxquels l'étudiant est affecté, si le Course Service propose cette fonctionnalité.

Le Student Service attend la réponse avant d'afficher les informations demandées.

- 200 OK : les cours de l'étudiant sont récupérés.
- 404 Not Found : aucun étudiant ou aucune affectation correspondante n'est trouvé.
- 503 Service Unavailable : le Course Service est indisponible.

B. Consultation auprès de l'Attendance Service

Le Student Service peut appeler l'Attendance Service via :

GET /api/attendance/student/{studentId}

Cet appel permet de récupérer l'historique des présences et des absences d'un étudiant afin de présenter une vue globale de son parcours académique.

- 200 OK : les informations de présence sont récupérées.
- 404 Not Found : aucun historique correspondant n'est trouvé.
- 503 Service Unavailable : l'Attendance Service est indisponible.

Le mode synchrone permet de récupérer des informations actualisées auprès des services propriétaires de ces données. Le Student Service ne stocke pas les cours ni les présences : il se contente de les consulter.

- **2. Asynchrone via RabbitMQ**

  Lorsqu'un étudiant est créé, modifié ou désactivé, le Student Service peut publier un événement dans RabbitMQ pour informer les autres microservices.

  Exemple : StudentCreated, contenant les informations nécessaires : studentId, firstName, lastName, email, groupId et status.

  L'Attendance Service peut consommer cet événement afin de connaître les nouveaux étudiants et de préparer leur prise en charge dans le suivi des présences.

  D'autres événements peuvent être publiés :

  - StudentUpdated : lorsqu'une information importante du profil est modifiée.
  - StudentDeactivated : lorsqu'un étudiant quitte l'établissement ou que son compte est désactivé.

  Les autres services peuvent alors actualiser les informations locales nécessaires ou empêcher de nouvelles opérations pour un étudiant désactivé.

  Le mode asynchrone est adapté, car la création ou la modification d'un profil étudiant ne doit pas attendre que tous les autres microservices traitent l'événement. Si un consommateur est indisponible, le message reste dans RabbitMQ en attente de traitement

3. Appelants du Student Service

Le Student Service peut être appelé par :

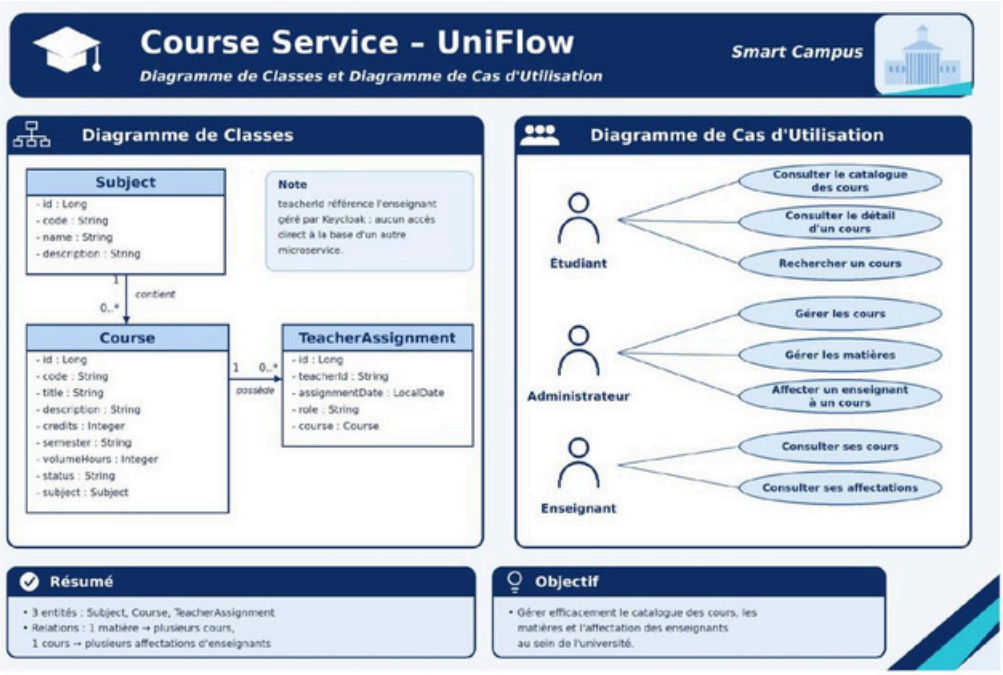
- Frontend Angular : les administrateurs peuvent créer, modifier, supprimer ou consulter les profils étudiants ; les étudiants peuvent consulter et mettre à jour certaines informations personnelles.
- Attendance Service : il peut consulter les informations d'un étudiant via OpenFeign pour vérifier son existence et récupérer ses données de référence.
- Course Service : il peut consulter les informations des étudiants via OpenFeign afin de vérifier leur existence avant de les affecter à des cours.
- Timetable Service : il peut consulter les informations des étudiants ou de leurs groupes pour afficher les emplois du temps correspondants, selon les besoins du projet.

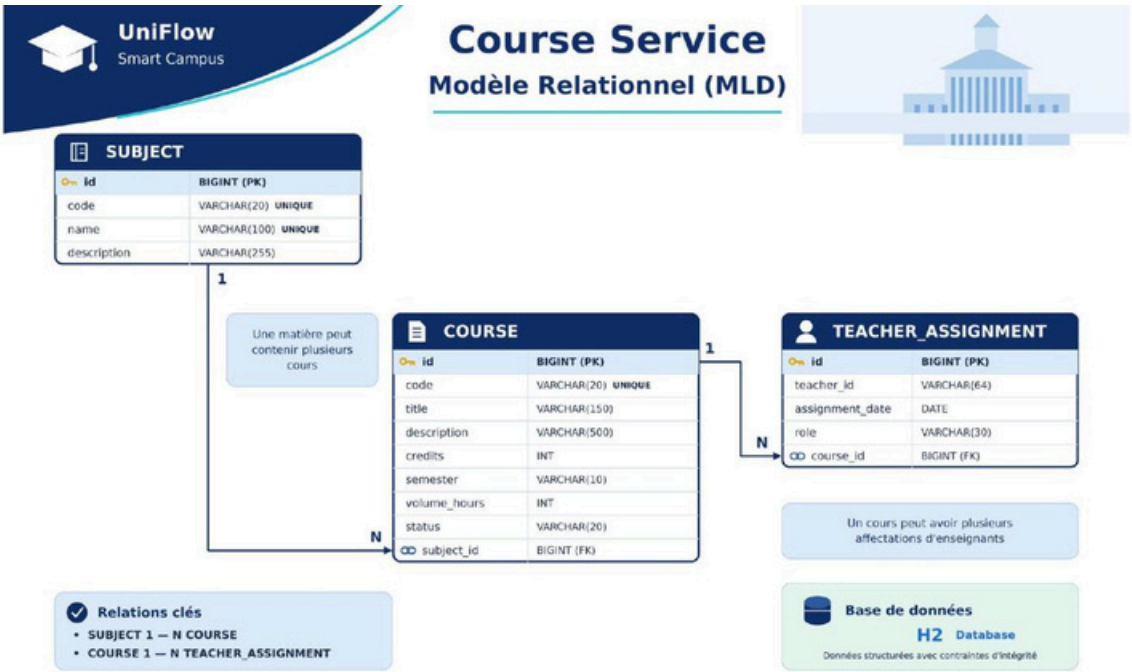
Les utilisateurs accèdent aux fonctionnalités via le frontend Angular, en passant par l'API Gateway, selon les routes configurées dans le projet.

4. Projet Spring Boot

- Dossier: Backend/Student-Service
- Nom de l'application : student-service
- Port : à confirmer.8081
- Dépendances : Spring Web, Spring Data JPA, Validation, pilote MySQL, Eureka Discovery Client, Spring AMQP (RabbitMQ), springdoc-openapi et éventuellement Spring Cloud OpenFeign.
- Branche de travail : feature/student-service

9.2 Course Service

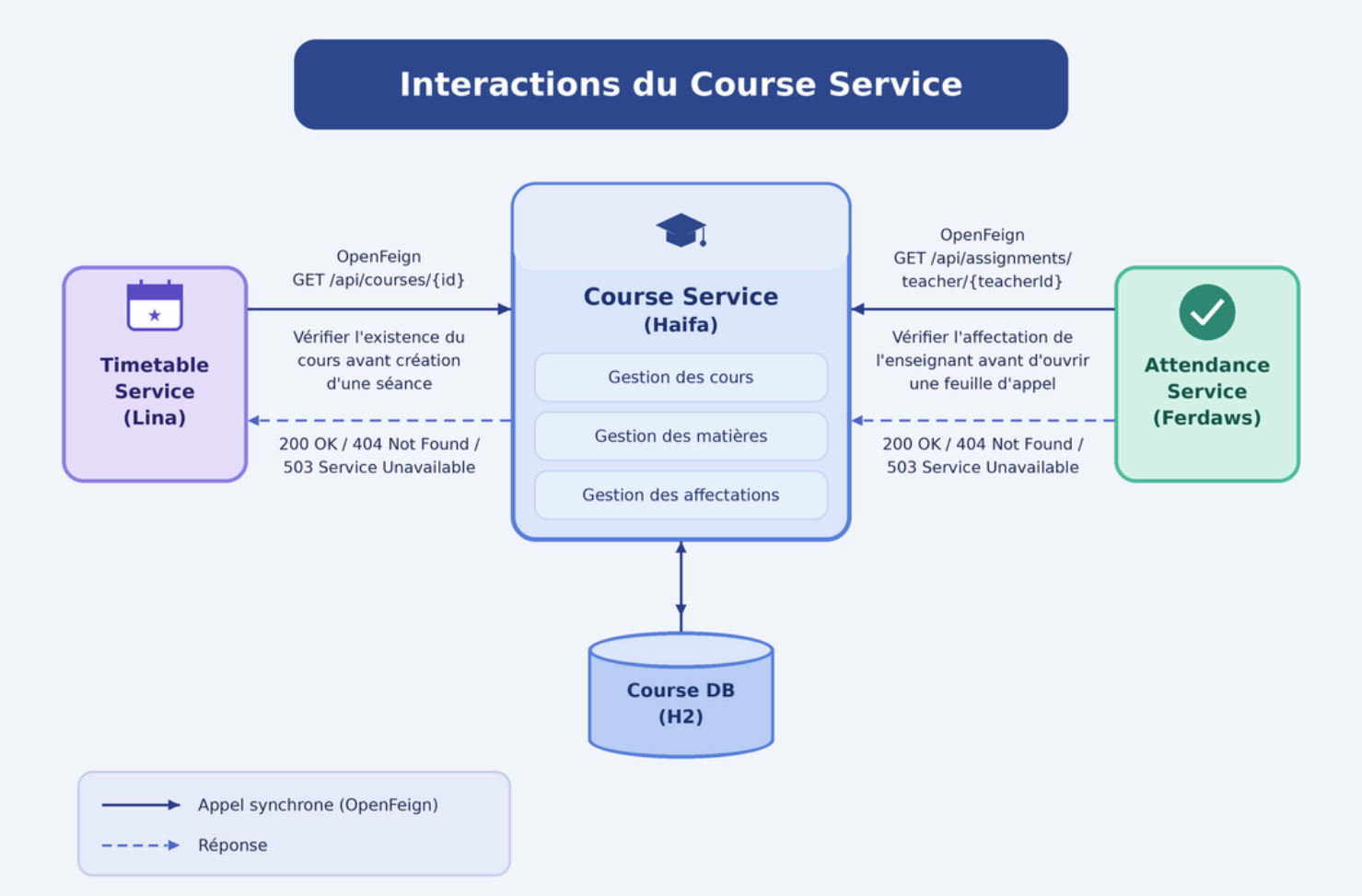




Timetable Service : contrat d'API

Méthode et URL	Rôle	Description	Codes
GET /api/timetables	ADMIN, TEACHER	Liste des emplois du temps	200
GET /api/timetables/{id}	ADMIN, TEACHER, STUDENT	Détail d'un emploi du temps	200, 404
GET /api/timetables/student/{studentId}	ADMIN, STUDENT	Emploi du temps d'un étudiant	200, 404
POST /api/timetables	ADMIN	Créer un emploi du temps	201, 400, 409
PUT /api/timetables/{id}	ADMIN	Modifier un emploi du temps	200, 400, 404, 409
DELETE /api/timetables/{id}	ADMIN	Supprimer un emploi du temps	204, 404, 409
GET /api/sessions/{id}	ADMIN, TEACHER, STUDENT	Détail d'une séance	200, 404
GET /api/sessions/teacher/{teacherId}	ADMIN, TEACHER	Planning de séances d'un enseignant	200, 404
POST /api/sessions	ADMIN	Créer une séance	201, 400, 404, 409, 503
PUT /api/sessions/{id}	ADMIN	Modifier une séance	200, 400, 404, 409, 503
DELETE /api/sessions/{id}	ADMIN	Supprimer une séance	204, 404
GET /api/time-slots	ADMIN, TEACHER, STUDENT	Liste des créneaux	200
POST /api/time-slots	ADMIN	Ajouter un créneau	201, 400, 409
DELETE /api/time-slots/{id}	ADMIN	Supprimer un créneau	204, 404, 409

Interactions



1. Synchrone depuis TimeTabme (OpenFeign)

Lors de la création ou de la planification d'une séance, le Timetable Service appelle le Course Service via GET /api/courses/{courseId}, afin de récupérer et vérifier les informations du cours : nom, code, matière, enseignant affecté, groupe et informations académiques nécessaires.

Le Timetable Service attend la réponse avant de poursuivre la planification de la séance.

- 200 OK : le cours existe ; ses informations sont récupérées et la planification peut continuer.
- 404 Not Found : le cours n'existe pas ; la création de la séance est refusée.
- 409 Conflict : le cours n'est pas disponible pour la planification selon son état (par exemple, cours désactivé).
- 503 Service Unavailable : le Course Service est indisponible ; la planification est temporairement interrompue.

Le mode synchrone est nécessaire, car le Timetable Service doit disposer des informations d'un cours valide avant de programmer une séance. Le Course Service reste responsable de la gestion des cours, des matières et des enseignants affectés, tandis que le Timetable Service gère les horaires et les séances.

2. Asynchrone via RabbitMQ

Lorsqu'un cours est créé, modifié ou désactivé, le Course Service publie un événement dans RabbitMQ.

Exemple : CourseUpdated, contenant les informations nécessaires : courseId, courseName, subjectId, teacherId, groupId et status.

Le Timetable Service et l'Attendance Service peuvent consommer cet événement afin de mettre à jour les informations locales nécessaires à leurs traitements.

Par exemple :

- Le Timetable Service actualise les informations du cours utilisées dans la planification.
- L'Attendance Service actualise les informations descriptives associées au cours, si elles sont conservées localement.

Le mode asynchrone est adapté, car la modification d'un cours ne doit pas attendre la réponse des autres microservices. Si l'un des consommateurs est indisponible, le message reste dans le broker en attente de traitement.

3. Appelants du course Service

Le Course Service peut être appelé par :

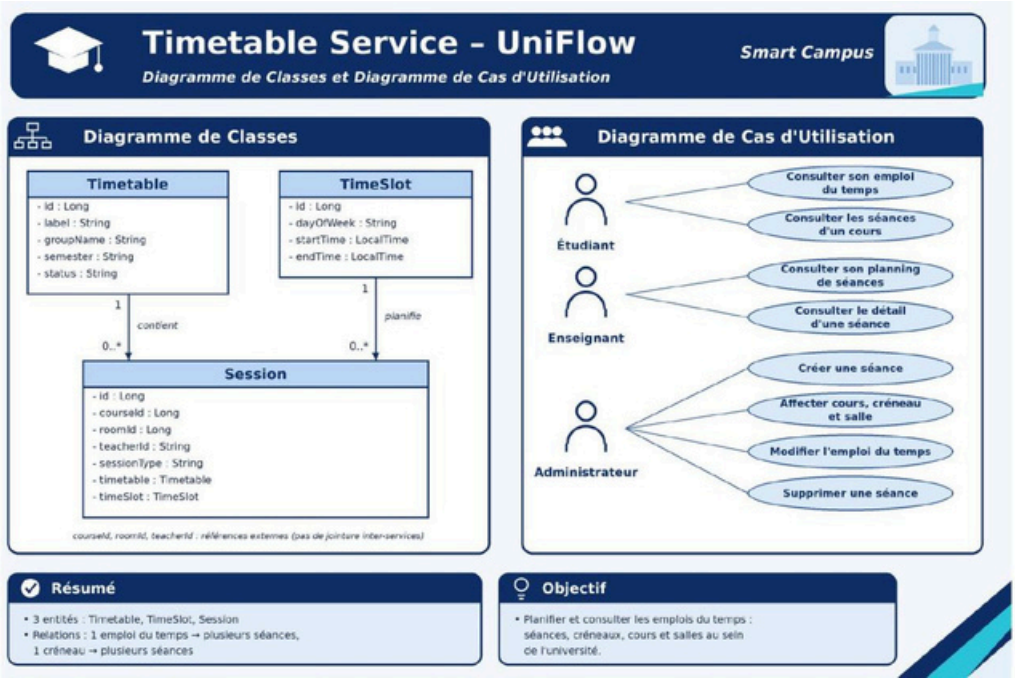
- Frontend Angular : les administrateurs peuvent créer, modifier et consulter les cours ; les enseignants peuvent consulter les cours qui leur sont affectés.
- Timetable Service : il récupère les informations des cours et des enseignants pour organiser les séances.
- Attendance Service : il récupère les informations des cours pour associer les présences aux matières concernées.
- Autres microservices autorisés : ils peuvent consulter les informations des cours selon leurs besoins fonctionnels.

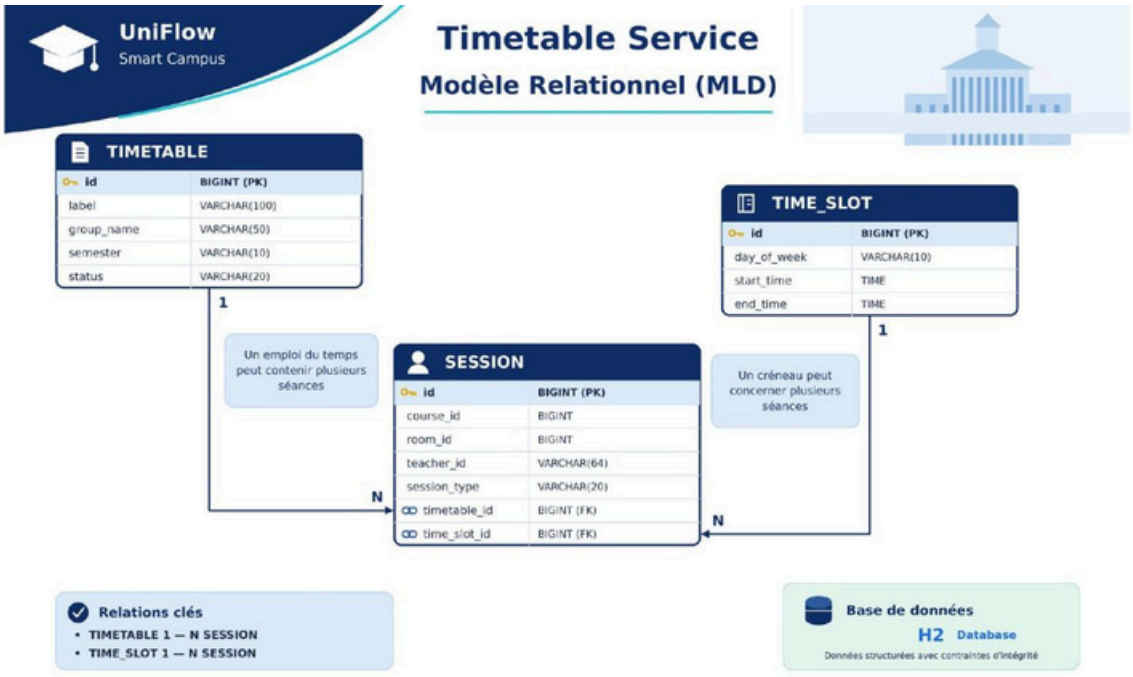
Les utilisateurs accèdent aux fonctionnalités via le frontend Angular, en passant par l'API Gateway, selon les routes configurées dans le projet.

4. Projet Spring Boot

- Dossier : Backend/Course-Service
- Nom de l'application : course-service
- Port : 8082
- Dépendances : Spring Web, Spring Data JPA, Validation, idem avec H2, Eureka Discovery Client, Spring AMQP (RabbitMQ) et springdoc-openapi
- Branche de travail : feature/course-service

9.3 Timetable Service

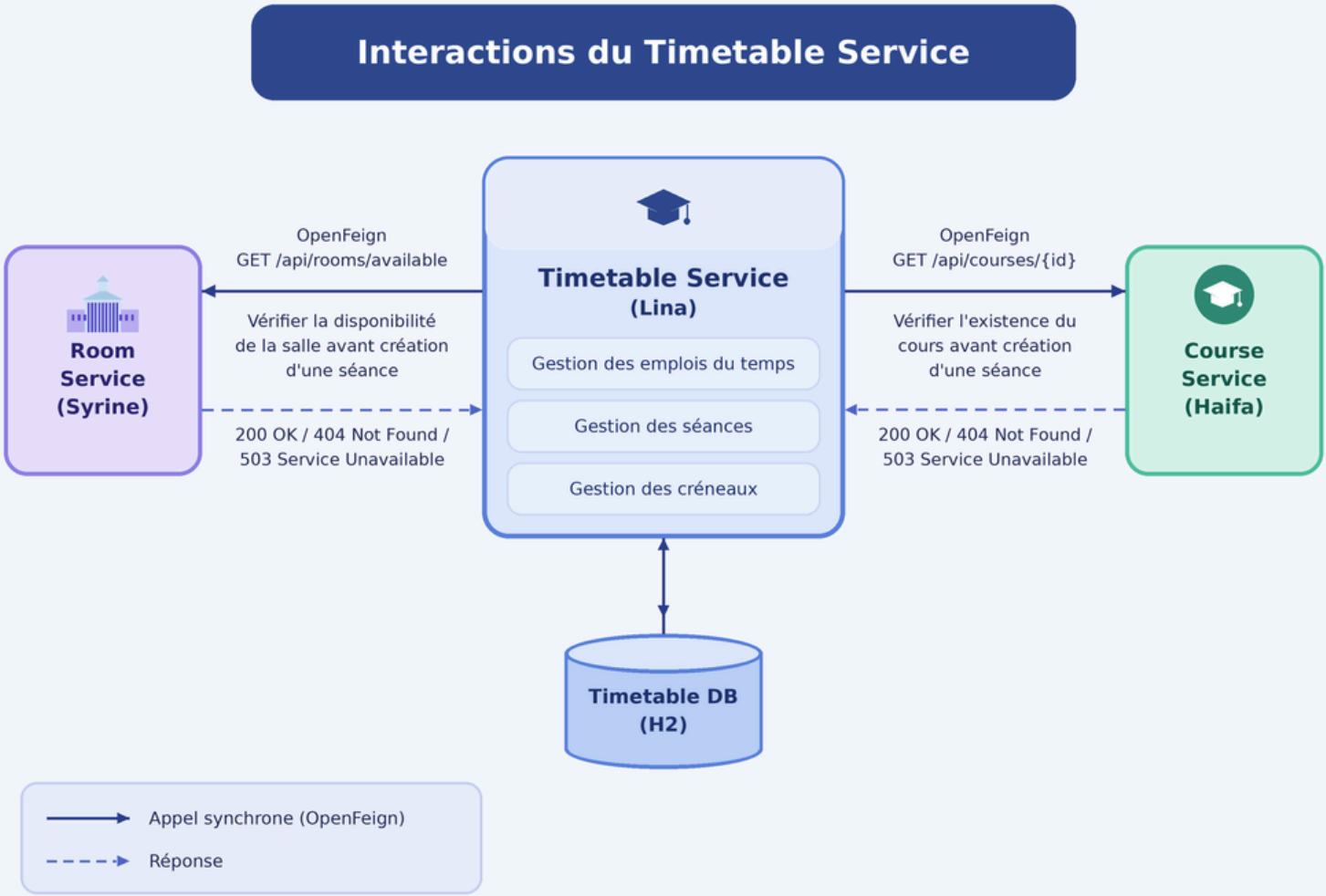




Timetable Service : contrat d'API

Méthode et URL	Rôle	Description	Codes
GET /api/timetables	ADMIN, TEACHER	Liste des emplois du temps	200
GET /api/timetables/{id}	ADMIN, TEACHER, STUDENT	Détail d'un emploi du temps	200, 404
GET /api/timetables/student/{studentId}	ADMIN, STUDENT	Emploi du temps d'un étudiant	200, 404
POST /api/timetables	ADMIN	Créer un emploi du temps	201, 400, 409
PUT /api/timetables/{id}	ADMIN	Modifier un emploi du temps	200, 400, 404, 409
DELETE /api/timetables/{id}	ADMIN	Supprimer un emploi du temps	204, 404, 409
GET /api/sessions/{id}	ADMIN, TEACHER, STUDENT	Détail d'une séance	200, 404
GET /api/sessions/teacher/{teacherId}	ADMIN, TEACHER	Planning de séances d'un enseignant	200, 404
POST /api/sessions	ADMIN	Créer une séance	201, 400, 404, 409, 503
PUT /api/sessions/{id}	ADMIN	Modifier une séance	200, 400, 404, 409, 503
DELETE /api/sessions/{id}	ADMIN	Supprimer une séance	204, 404
GET /api/time-slots	ADMIN, TEACHER, STUDENT	Liste des créneaux	200
POST /api/time-slots	ADMIN	Ajouter un créneau	201, 400, 409
DELETE /api/time-slots/{id}	ADMIN	Supprimer un créneau	204, 404, 409

Interactions



1. Synchrone vers Course Service et Room Service (OpenFeign)

Lors de la création d'une séance, le Timetable Service vérifie l'existence du cours et la disponibilité de la salle avant de confirmer la planification.

A. Vérification auprès du Course Service

Appel via GET /api/courses/{courseId} pour récupérer les informations du cours (nom, enseignant et groupe).

- 200 OK : cours trouvé.
- 404 Not Found : cours inexistant.
- 409 Conflict : cours non planifiable.
- 503 Service Unavailable : service indisponible.

B. Réservation auprès du Room Service

Appel via POST /api/rooms/reservations, avec roomId, date, startTime, endTime et sessionId.

- 201 Created : réservation confirmée.
- 404 Not Found : salle inexistante.
- 409 Conflict : salle déjà réservée.
- 503 Service Unavailable : service indisponible.

La communication synchrone permet de vérifier les informations nécessaires avant de confirmer une séance et d'éviter les conflits de réservation.

2. Asynchrone via RabbitMQ

Lorsqu'une séance est créée, modifiée ou annulée, le Timetable Service publie un événement RabbitMQ :

- SessionScheduled : création d'une séance.
- SessionUpdated : modification d'une séance.
- SessionCancelled : annulation d'une séance.

L'Attendance Service consomme ces événements pour actualiser les séances et adapter la gestion des présences.

Cette communication asynchrone évite de bloquer le Timetable Service si l'Attendance Service est temporairement indisponible.

3. Appelants du TimeTable Service

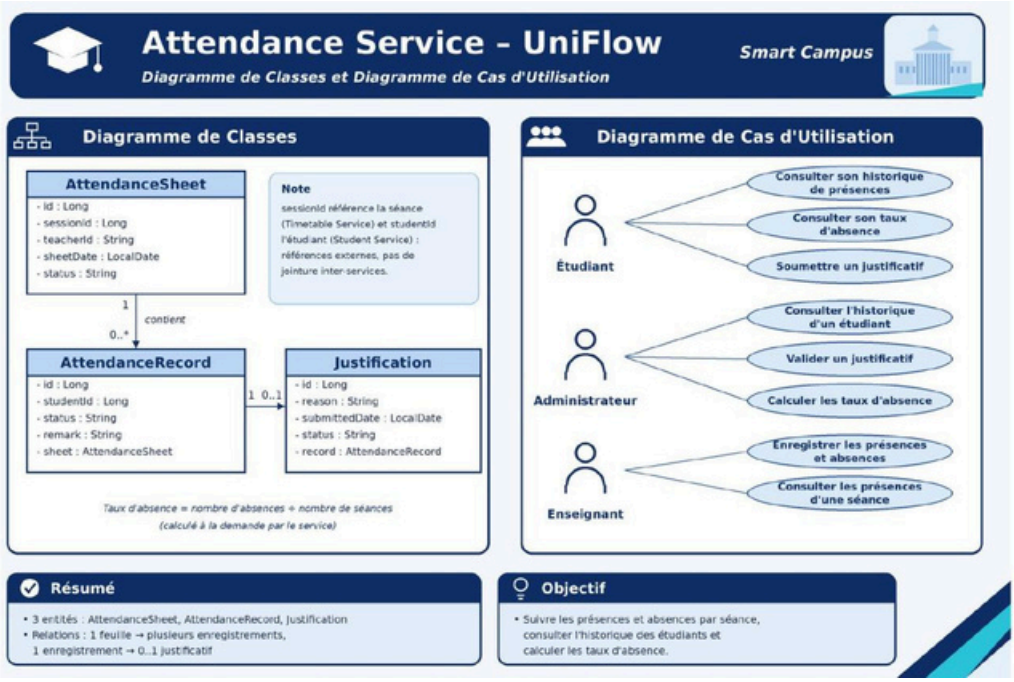
- Frontend Angular : création, modification, suppression et consultation des emplois du temps.
- Attendance Service : vérification des séances via OpenFeign.
- Course Service : récupération des informations des cours.
- Room Service : réservation des salles.

Les requêtes du frontend passent par l'API Gateway.

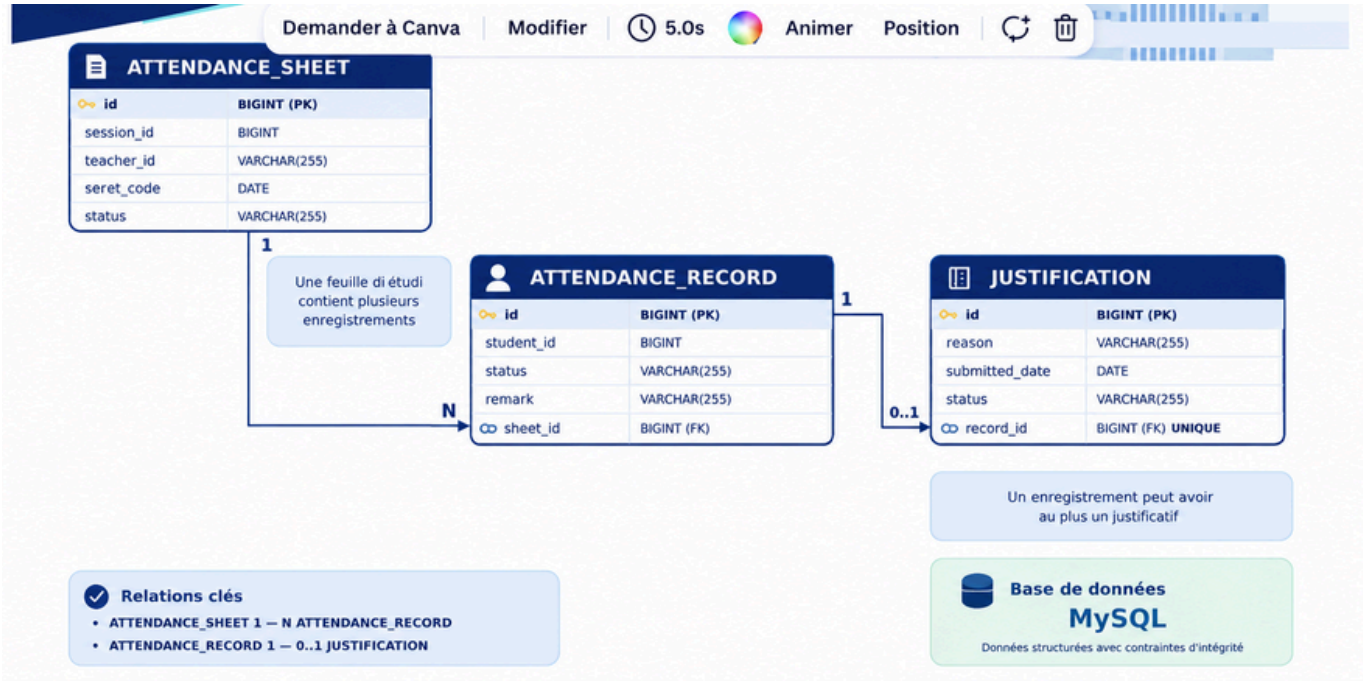
4. Projet Spring Boot

- Dossier: Backend/Timetable-Service
- Nom de l'application : timetable-service
- Port : 8083
- Dépendances : Spring Web, Spring Data JPA, Validation, MySQL, OpenFeign, Eureka Client, RabbitMQ et Springdoc OpenAPI.
- Branche de travail : feature/timetable-service

9.4 Attendance Service



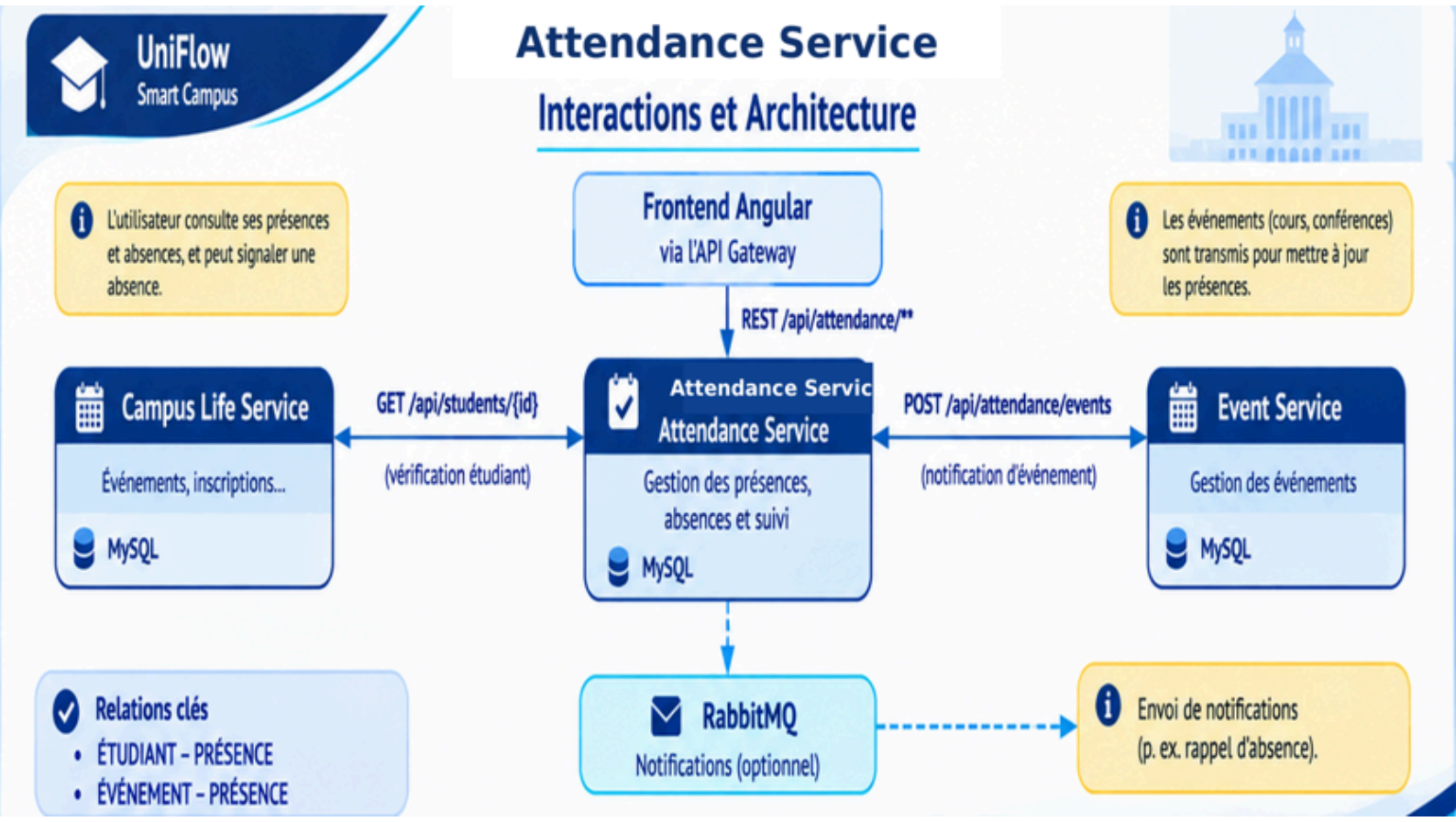
Modèle de données et choix de la base



API REST

Méthode	URL	Rôle	Codes
GET	/api/attendance	ADMIN, TEACHER	200
GET	GET /api/attendance/{id}	ADMIN, TEACHER, STUDENT	200, 404
POST	/api/attendance	TEACHER	201, 400, 404, 409
PUT	/api/attendance/{id}	TEACHER	200, 400, 404
DELETE	/api/attendance/{id}	ADMIN	204, 404
GET	/api/attendance/student/{studentId}	STUDENT, ADMIN	200, 404
GET	/api/attendance/report	ADMIN, TEACHER	200, 400
POST	/api/attendance/absence	STUDENT	201, 400, 409

Interactions



1. Synchrone vers TimeTable Service (OpenFeign)

Lorsqu'un enseignant souhaite démarrer une session de présence, l'Attendance Service appelle le Timetable Service via GET /api/timetable/sessions/{sessionId}, afin de vérifier que la séance existe et de récupérer ses informations : courseId, teacherId, date, heure de début, heure de fin et groupe concerné.

L'Attendance Service attend la réponse avant de permettre l'ouverture de la session de présence.

200 OK : la séance existe et ses informations sont récupérées ; la session de présence peut être ouverte.

404 Not Found : la séance n'existe pas ; l'ouverture de la session de présence est refusée.

409 Conflict : la séance ne peut pas faire l'objet d'un appel (par exemple, séance annulée ou déjà terminée) ; l'opération est refusée.

503 Service Unavailable : le Timetable Service est indisponible ; l'ouverture est temporairement impossible.

Le mode synchrone est nécessaire, car l'Attendance Service doit vérifier l'existence de la séance planifiée avant d'enregistrer les présences. Il ne doit pas créer son propre emploi du temps.

2. Asynchrone via RabbitMQ

Lorsqu'une absence est enregistrée, l'Attendance Service publie un événement StudentMarkedAbsent dans RabbitMQ, contenant les informations nécessaires : recordId, studentId, sessionId, courseId, status et recordedAt.

Le service de notification consomme cet événement afin d'envoyer une notification à l'étudiant concerné pour l'informer de son absence.

L'Attendance Service n'attend pas la confirmation de réception pour finaliser l'enregistrement de l'absence.

Le mode asynchrone est adapté, car l'enregistrement d'une absence ne doit pas dépendre de la disponibilité du service de notification. Si ce dernier est indisponible, le message reste dans le broker en attente de traitement.

3. Appelants du Attendance Service

L'Attendance Service peut être appelé par :

Frontend Angular : les enseignants peuvent effectuer l'appel et gérer les présences ; les étudiants peuvent consulter leur historique et leurs absences.

Timetable Service : l'Attendance Service consulte les séances planifiées via OpenFeign.

Student Service : l'Attendance Service peut récupérer les informations ou vérifier les étudiants concernés, si nécessaire.

Service de notification : il consomme les événements d'absence publiés via RabbitMQ.

Les utilisateurs accèdent aux fonctionnalités via le frontend Angular, en passant par l'API Gateway, selon les routes configurées dans le projet

4. Projet Spring Boot

Dossier : Backend/Attendance-Service

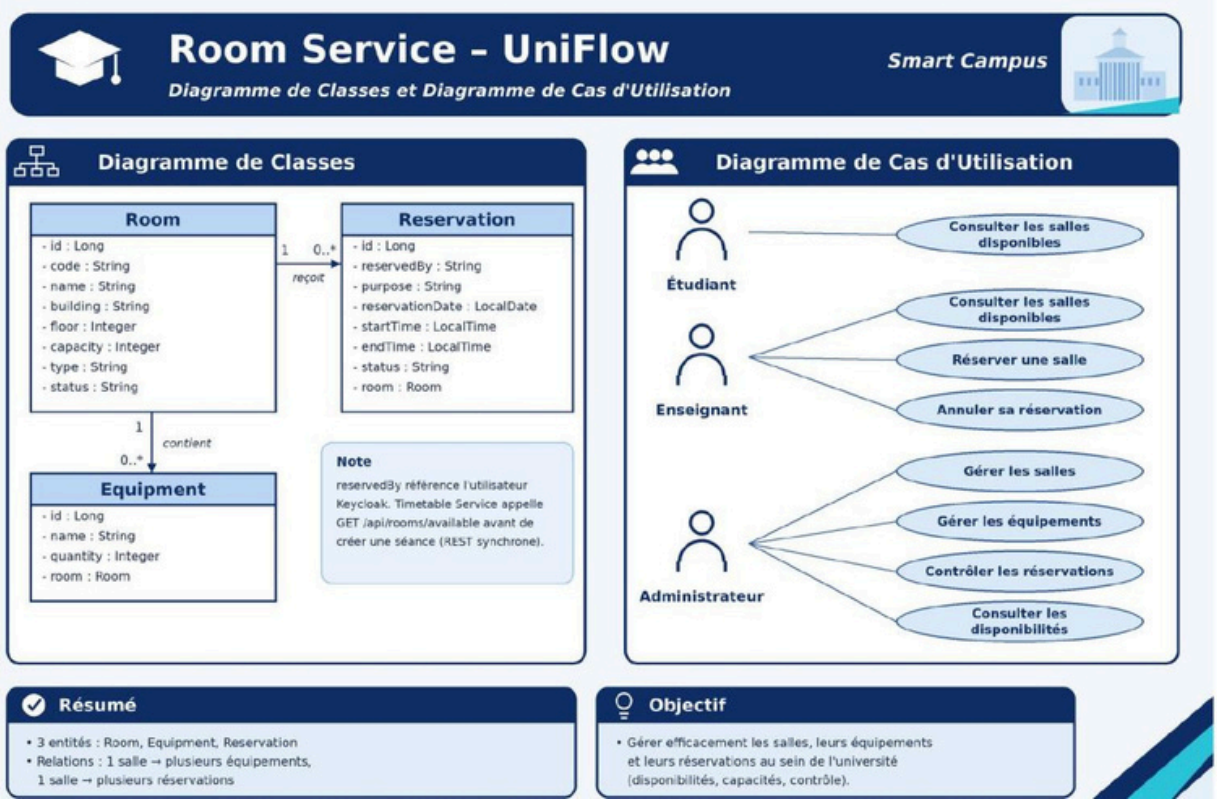
Nom de l'application : attendance-service

Port : 8084

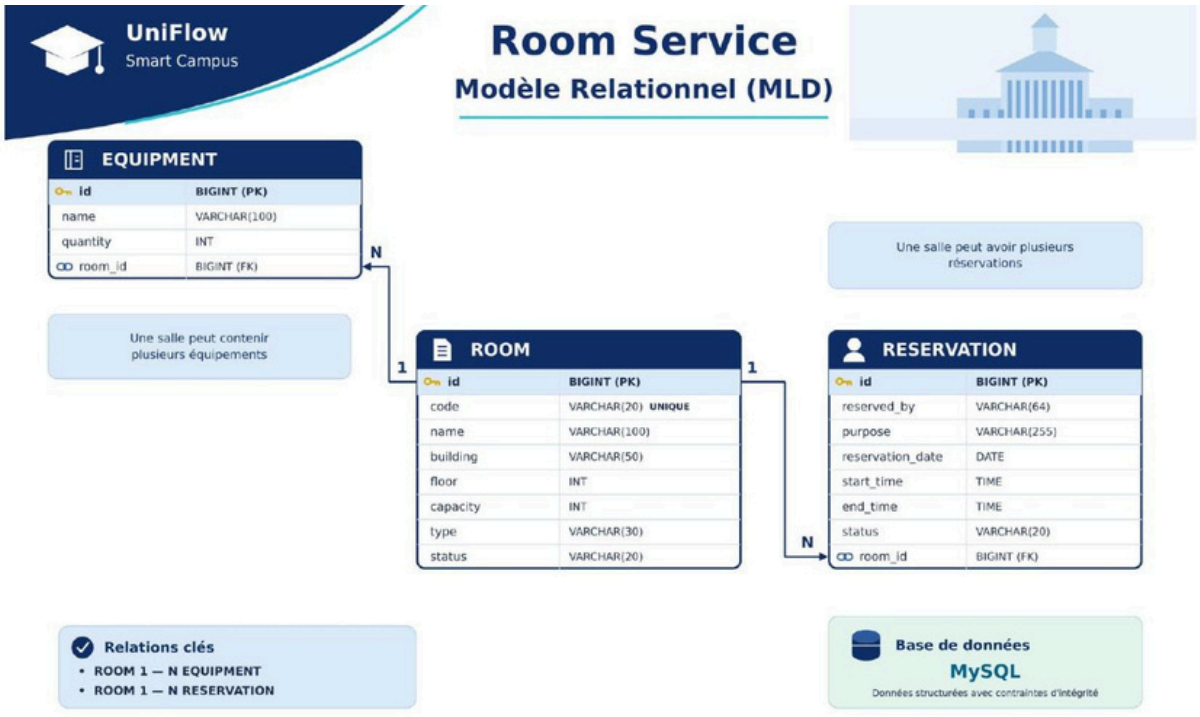
Dépendances : Spring Web, Spring Data JPA, Validation, pilote MySQL, OpenFeign, Eureka Discovery Client, Spring AMQP (RabbitMQ) et springdoc-openapi

Branche de travail : feature/attendance-service

9.5 Room Service



Modèle de données et choix de la base

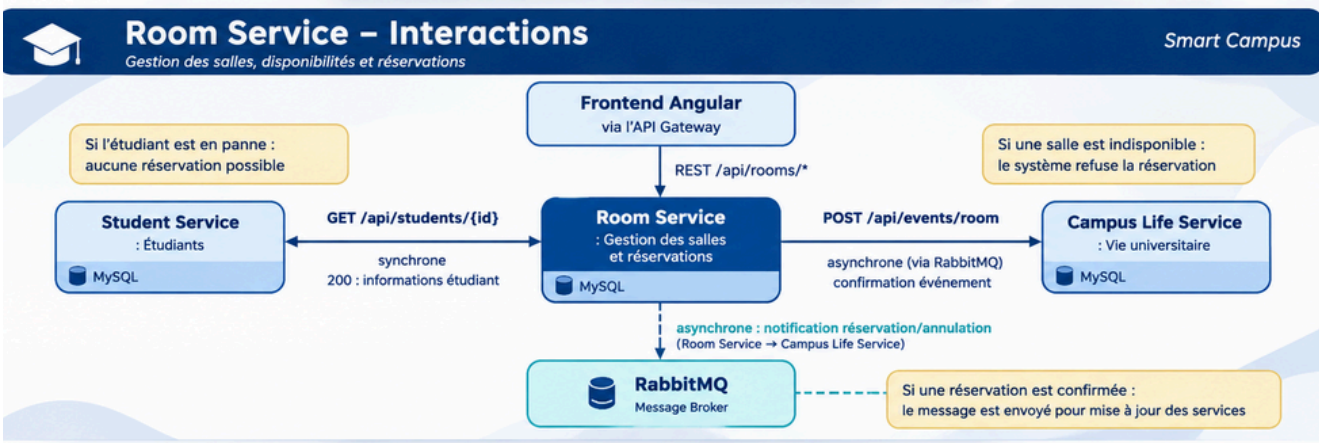


MySQL est retenu comme système de gestion de base de données afin de garantir l'intégrité et la cohérence des données relatives aux salles et aux réservations. Il permet notamment de contrôler la disponibilité des salles, d'éviter les conflits de réservation et d'assurer la fiabilité des opérations grâce à la gestion des transactions.

API REST

Méthode et URL	Rôle	Description	Codes
GET /api/rooms	Tous	Lister toutes les salles	200 , 404
GET /api/rooms/{id}	Tous	Détail d'une salle	200, 404
POST /api/rooms	ADMIN	Ajouter une salle	201, 400
PUT /api/rooms/{id}	ADMIN	Modifier une salle	200 ,400
DELETE /api/rooms/{id}	ADMIN	Supprimer une salle	204, 404
GET /api/rooms/reservations	Tous	Lister des réservations	200, 404
POST /api/rooms/reservations	STUDENT	Créer une réservation	201, 400, 409
DELETE /api/rooms/reservations/{id}	STUDENT	Annuler une réservations	204 , 404

Interactions





1. Synchrone vers Room Service (OpenFeign)

Lors de la publication d'un événement nécessitant une salle, Campus Life appelle le Room Service via POST /api/rooms/reservations, en transmettant les informations nécessaires : roomId, date, heure de début, heure de fin, objet et organisateur. Campus Life attend la réponse avant de publier l'événement.

201 : la réservation de la salle est créée et l'événement peut être publié.

409 : la salle est déjà réservée sur ce créneau ; la publication de l'événement est refusée.

503 : le Room Service est indisponible ; l'événement n'est pas publié.

Le mode synchrone est nécessaire, car la publication de l'événement dépend directement de la disponibilité de la salle.

2. Asynchrone via RabbitMQ

Lorsqu'un événement est annulé, Campus Life publie le message EventCancelled (eventId, roomId, startDate, endDate) dans RabbitMQ, sans attendre de réponse.

Le Room Service consomme ce message et libère la salle correspondant à la réservation. Le mode asynchrone est adapté, car l'annulation de l'événement ne dépend pas de la réponse du Room Service. Si celui-ci est indisponible, le message reste dans le broker en attente de traitement.

3. Appelants du Room Service

Le Room Service peut être appelé par Campus Life pour gérer les réservations de salles. Les utilisateurs peuvent également accéder à ses fonctionnalités via le frontend Angular, en passant par l'API Gateway, selon les routes configurées dans le projet.

4. Projet Spring Boot

Dossier Backend/Room-Service

Nom de l'application : room-service

Port : 8085.

Dépendances : Spring Web, Spring Data JPA, Validation, pilote MySQL, Eureka Discovery Client et springdoc-openapi

Branche de travail : feature/room-service.

9.6 Campus Life Service (Farah)

Responsabilité

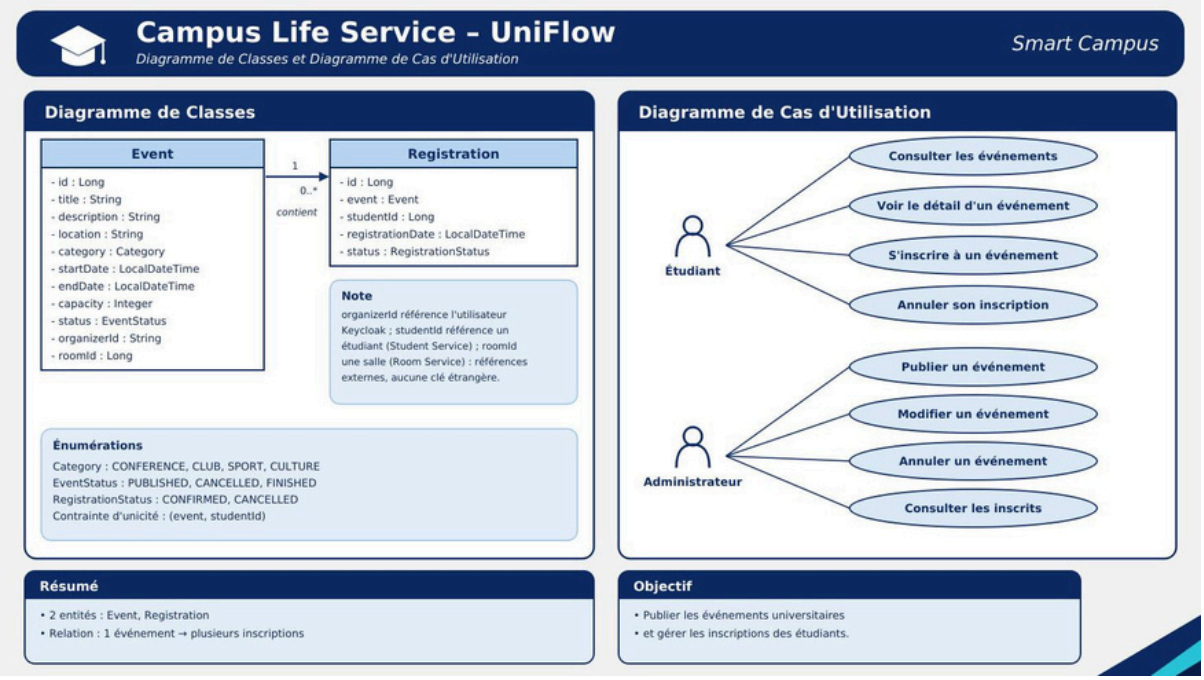
Publieretgérerlesévénementsuniversitaires, et gérer les inscriptions des étudiants.

Entités

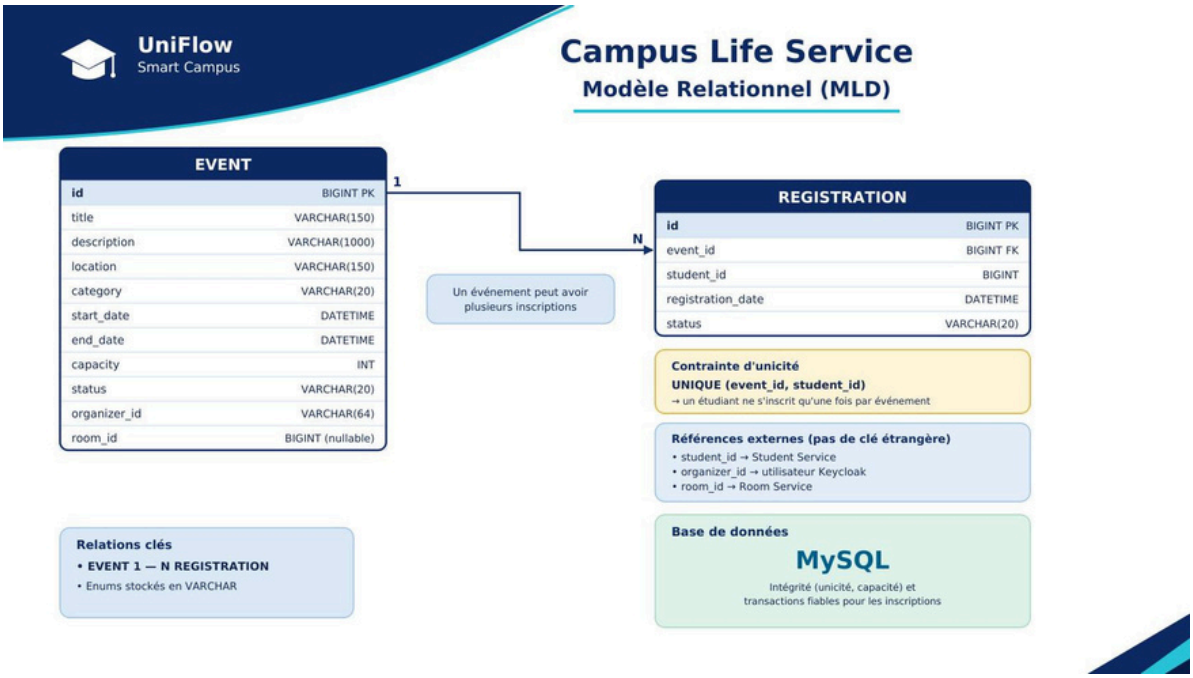
Entité	Attributs
Event	id (Long), title, description, location (String), category (CONFERENCE, CLUB, SPORT, CULTURE), startDate, endDate (LocalDateTime), capacity (Integer), status

Entité	Attributs
	(PUBLISHED, CANCELLED, FINISHED), organizerId (String, référence Keycloak), roomId (Long, optionnel, référence Room)
Registration	id (Long), event (Event), studentId (Long, référence Student), registrationDate (LocalDateTime), status (CONFIRMED, CANCELLED). Unicité sur (event, studentI d)

Relation : un Event contient0àplusieursRegistration ; chaqueRegistration concerne un seul Event. studentId, organizerIdetroomId sont de simples références, sans clé étrangère.



Modèle de données et choix de la base

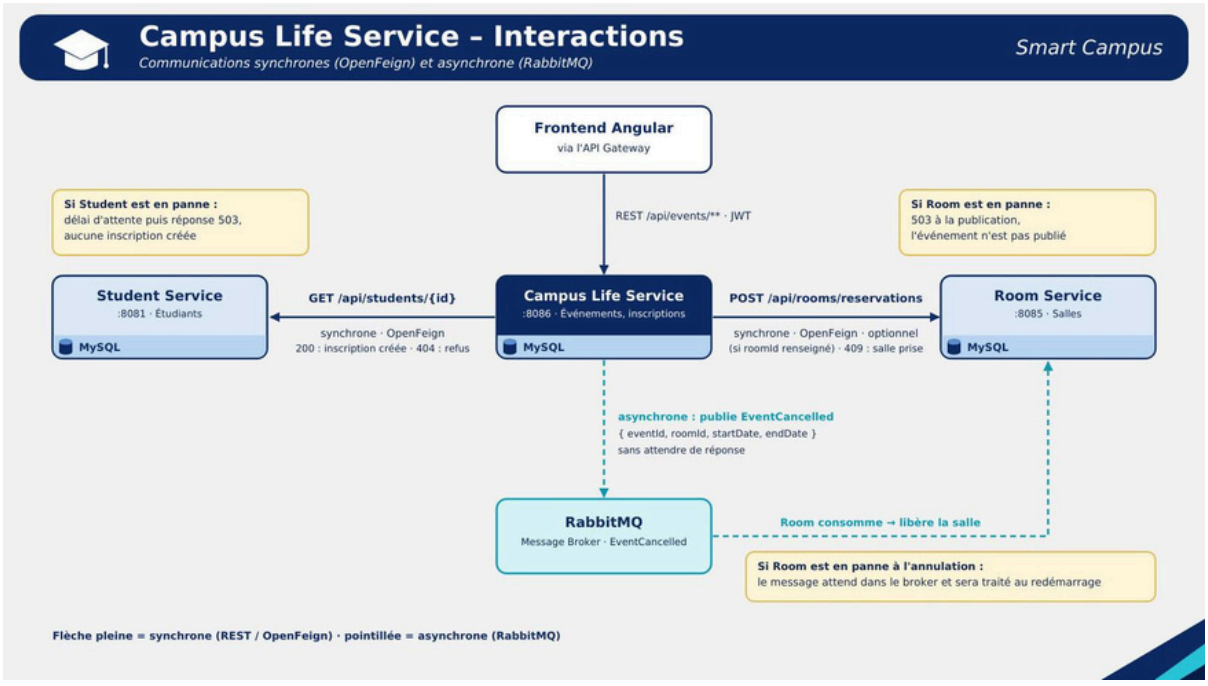


MySQL est retenu pour ses contraintes d'intégrité (unicité d'une inscription par étudiant et par événement, respect de la capacité) et ses transactions fiables pour les inscriptions.

API REST

Méthode et URL	Rôle	Description	Codes
GET /api/events	Tous	Liste des événements (filtres category, date)	200
GET /api/events/{id}	Tous	Détail d'un événement	200, 404
POST /api/events	ADMIN	Publier un événement ; réserve la salle si roomId est renseigné	201, 400, 409, 503
PUT /api/events/{id}	ADMIN	Modifier un événement	200, 400, 404
DELETE /api/events/{id}	ADMIN	Annulation logique (statut CANCELLED) et publication de EventCancelled	204, 404
GET /api/events/{id}/registrations	ADMIN	Liste des inscrits	200, 404
POST /api/events/{id}/registrations	STUDENT	S'inscrire à un événement	201, 400, 404, 409, 503
DELETE /api/events/{id}/registrations/{studentId}	STUDENT	Annuler son inscription	204, 404
GET /api/registrations/student/{studentId}	STUDENT	Inscriptions d'un étudiant	200

Interactions



1. Synchrone vers Student (OpenFeign)
À l'inscription, Campus Life appelle GET/api/students/{id} et attend la réponse. 200 : l'inscription est créée ; 404 : refus. Le mode synchroneestnécessaire car l'inscription dépend immédiatement de l'existence de l'étudiant. Si Studentestindisponible : délai d'attente puis réponse contrôlée 503, sans créer d'inscription.

2. Synchrone (optionnel) vers Room
À la publication d'un événement avecunesalle,Campus Life appelle POST /api/rooms/reservations (roomId, date,heuresdedébut et de fin, objet, organisateur). 201 : la réservation est créée ; 409 : la salle estdéjàpriseet l'événement n'est pas publié ; Room

indisponible : 503. Le mode synchrone est nécessaire car la publication dépend de la disponibilité de la salle.

3. Asynchrone via RabbitMQ
À l'annulation d'un événement,Campus Life publie le message EventCancelled (eventId, roomId, startDate, endDate) dans le broker, sans attendre de réponse. Room le consomme et libère la salle. Le mode asynchrone estadapté car l'annulation ne dépend pas de la réponse de Room ; si Room est indisponible, le message attend dans le broker.

Appelants de Campus Life : lefrontend Angular, via l'API Gateway.

Projet Spring Boot

Dossier Backend/Campus-Life-Service, nom d'application campus-life-service, port 8086. Dépendances : Spring Web, Spring Data JPA, Validation, pilote MySQL, Eureka Discovery Client, springdoc-openapi. Branche de travail : feature/campus-life.